

Venus Token Converter Cap Mechanism Update Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/05/07 - 2026/05/08 (YYYY/MM/DD)

Overall Risk: **Low**

Project Name	Venus Token Converter Cap Mechanism Update Audit Report
Github	https://github.com/VenusProtocol/protocol-reserve/pull/159/changes/ac8509318393658b0145a9d643933ea1ef597ec0

Smart contracts list:

No.	Contract Name	Verdict	Details
1	TokenBuyback.sol	Low	[L01] [L02] [I01] [I02] [I03] [I04] [I05] [I06]
2	ITokenBuyback.sol	Green	

Findings

[L01] Fixed window cap in `_enforceCapAndDetectSlippage` for `dailyCapUsd`

Contract TokenBuyback.sol

Severity Level Low

Description

Description

The current documentation for `dailyCapUsd` is misaligned with its behavior. The comment on `dailyCapUsd` states that consumption is bounded within a rolling 24h window, and the NatSpec on `_enforceCapAndDetectSlippage` describes the function as enforcing the daily cap.

JavaScript

```
/// @notice Maximum cumulative USD value of tokenIn consumed via
executeBuyback
/// within a rolling 24h window. Bounds blast radius if the
operator key
/// is compromised. Stored 1e18-scaled.
uint256 public dailyCapUsd;
```

A true rolling 24h bound would require $\text{consumption}(t - 24h, t) \leq \text{dailyCapUsd}$ for every t , but the implementation only guarantees $\text{consumption}(\text{windowStart}, \text{windowStart} + \text{WINDOW}) \leq \text{dailyCapUsd}$ for the current fixed window.

An operator sizing `dailyCapUsd` against the maximum tolerable 24h loss could therefore underestimate the actual `dailyCapUsd` required

Recommendation

- Consider a documentation fix to update the comment on `dailyCapUsd` and the NatSpec on `_enforceCapAndDetectSlippage` to state explicitly that the cap uses fixed 24h windows from `windowStart`
- If not, if the implementation is required for a true rolling bound: replace the fixed window counter with a sliding window accumulator that records (timestamp, `usdIn`) contributions and, on each `executeBuyback` call, retrieves entries older than `WINDOW` before checking that $\text{sum}(\text{active entries}) + \text{usdIn} \leq \text{dailyCapUsd}$

Status

Partially Fixed. The leaky-bucket implementation in commit [ad790bd](#) is a meaningful improvement over the original tumbling-window design — the boundary-moment instantaneous 2× burst is eliminated. However, the updated NatSpec introduces an incorrect claim. Both `_enforceCapAndDetectSlippage` and `usdConsumedInWindow` now state:

JavaScript

```
"any rolling 24h interval's total consumption is bounded by dailyCapUsd"
```

This is mathematically false for a leaky-bucket accumulator. Counter-example with `dailyCapUsd = $30,000`:

```
JavaScript
- t = 0h: consume $30,000 → bucket full
- t = 23h: decay leaves 1/24 × $30,000 = $1,250; up to $28,750 more can be consumed
- Total consumption in the [0h, 23h] window: $58,750 ≈ 2× dailyCapUsd
```

The leaky bucket's correct invariant is that the bucket level is bounded by `dailyCapUsd` at any instant, not that rolling 24h consumption is bounded by `dailyCapUsd`. The updated NatSpec overstates the guarantee and may cause operators to undersize the cap against their actual risk tolerance.

Recommendation: Update the NatSpec for `dailyCapUsd`, `usdConsumedInWindow`, and `_enforceCapAndDetectSlippage` to remove the incorrect "bounded by `dailyCapUsd`" claim and replace it with an accurate description, for example:

```
JavaScript
"The bucket level is bounded by dailyCapUsd at any instant. Due to linear decay, total consumption over any rolling 24h window may reach up to ~2× dailyCapUsd if large swaps are spaced ~23h apart. Operators should size dailyCapUsd as half the maximum tolerable 24h loss if a strict bound is required."
```

If a strict rolling 24h bound is required by design, consider replacing the leaky-bucket with a ring-buffer accumulator (e.g., 24 hourly slots) which provides a true sliding-window guarantee.

[L02] Storage Upgrade Gap Under-Allocated by One Slot in TokenBuyback	
Contract	TokenBuyback.sol
Severity Level	Low
Description	Description This fix introduces four new state variables (<code>dailyCapUsd</code>, <code>slippageEventUsd</code>, <code>usdConsumedInWindow</code>, <code>lastUpdate</code>) and reduces <code>__gap</code> from <code>uint256[48]</code> to <code>uint256[43]</code> — a reduction of 5. Since <code>WINDOW</code> is declared as internal constant and consumes no storage slot, only 4 new slots were actually added. The original storage layout totalled 50 slots (2 variables + gap 48); the updated layout totals 49 slots (6 variables + gap 43), a net decrease of 1.

JavaScript

```
// TokenBuyback - storage layout
uint256 public dailyCapUsd;           // new, 1 slot
uint256 public slippageEventUsd;      // new, 1 slot
uint256 public usdConsumedInWindow;   // new, 1 slot
uint256 public lastUpdate;           // new, 1 slot
uint256 internal constant WINDOW = 24 hours; // constant - no slot
consumed
uint256[43] private __gap; // should be uint256[44]
```

Root cause: perBlockCapUsd was temporarily introduced as a fifth storage variable during development, at which point __gap was correctly reduced to [43]. When perBlockCapUsd was removed in commit 0c90ac7, the gap was not restored to [44].

Recommendation Change __gap from uint256[43] to uint256[44] to restore the contract's own storage total to 50 slots.

JavaScript

```
uint256[44] private __gap;
```

Status Fixed as recommended in commit [3e9caae](#).

[I01] Missing bounds validation in TokenBuyBack configuration setters

Contract TokenBuyback.sol

Severity Level **Informational**

Description

Description

The TokenBuyBack contract exposes setDailyCapUsd, setSlippageEventUsd, and setPerBlockCapUsd, along with initialize, without enforcing any bounds on the supplied values. This permits several harmful configurations.

- Setting dailyCapUsd = 0, whether deliberate or by mistake, blocks all swaps involving assets with a non zero price, effectively bricking the contract.
- Setting dailyCapUsd = type(uint256).max silently disables the cap protection altogether.
- Setting slippageEventUsd = 0 causes every swap with non positive slippage to emit AbnormalSlippage, flooding monitoring systems with noise and obscuring genuine anomalies.

JavaScript

```
/// @notice Updates the daily USD cap on tokenIn consumption
```

```

    /// @param newCap New cap value (1e18-scaled)
    /// @custom:event DailyCapUpdated emits on success
    /// @custom:access Restricted by ACM
    function setDailyCapUsd(uint256 newCap) external override {
        _checkAccessAllowed("setDailyCapUsd(uint256)");
        emit DailyCapUpdated(dailyCapUsd, newCap);
        dailyCapUsd = newCap;
    }

    /// @notice Updates the per-block USD cap on tokenIn consumption
    /// @param newCap New cap value (1e18-scaled)
    /// @custom:event PerBlockCapUpdated emits on success
    /// @custom:access Restricted by ACM
    function setPerBlockCapUsd(uint256 newCap) external override {
        _checkAccessAllowed("setPerBlockCapUsd(uint256)");
        emit PerBlockCapUpdated(perBlockCapUsd, newCap);
        perBlockCapUsd = newCap;
    }

    /// @notice Updates the absolute USD slippage threshold above which
    AbnormalSlippage is emitted
    /// @param newThreshold New threshold value (1e18-scaled)
    /// @custom:event SlippageEventUsdUpdated emits on success
    /// @custom:access Restricted by ACM
    function setSlippageEventUsd(uint256 newThreshold) external override
    {
        _checkAccessAllowed("setSlippageEventUsd(uint256)");
        emit SlippageEventUsdUpdated(slippageEventUsd, newThreshold);
        slippageEventUsd = newThreshold;
    }

```

Recommendation Consider adding an `ensureNonzeroValue` style check to `setDailyCapUsd`, `setSlippageEventUsd`, and `setPerBlockCapUsd`, and apply the same validation in `initialize`

Status Fixed in [efcd0b7](#) and [0c90ac7](#). The fix correctly addresses all three reported scenarios:

- **Zero-value:** `ensureNonzeroValue` is now applied in `setDailyCapUsd`, `setSlippageEventUsd`, and `initialize`, preventing both a bricked contract and event-flooding from a zero `slippageEventUsd`.
- **perBlockCapUsd:** The entire per-block cap mechanism was removed in commit `0c90ac7`, eliminating the associated misconfiguration risk entirely.
- **type(uint256).max:** While no explicit upper-bound check was added, this falls within the scope of governance responsibility — both setters are ACM-gated and any change requires a governance proposal. This is consistent with the original recommendation, which only required `ensureNonzeroValue`-style validation.

[I02] Cap enforcement in executeBuyBack runs after the swap and transfers

Contract TokenBuyback.sol

Severity Level Informational

Description

Description

In TokenBuyBack.executeBuyBack, _enforceCapAndDetectSlippage is invoked only after the router call has executed, the BASE_ASSET safeTransfer to DESTINATION has occurred, and the assetsReserves state writes have been applied.

While reverts unwind state cleanly so no funds are lost, this ordering would cause every transaction that exceeds the cap to be fully executed before reverting via _enforceCapAndDetectSlippage, wasting operator gas on work that will be undone.

JavaScript

```
/// @notice Updates the daily USD cap on tokenIn consumption
/// @param newCap New cap value (1e18-scaled)
/// @custom:event DailyCapUpdated emits on success
/// @custom:access Restricted by ACM
function setDailyCapUsd(uint256 newCap) external override {
    _checkAccessAllowed("setDailyCapUsd(uint256)");
    emit DailyCapUpdated(dailyCapUsd, newCap);
    dailyCapUsd = newCap;
}

/// @notice Updates the per-block USD cap on tokenIn consumption
/// @param newCap New cap value (1e18-scaled)
/// @custom:event PerBlockCapUpdated emits on success
/// @custom:access Restricted by ACM
function setPerBlockCapUsd(uint256 newCap) external override {
    _checkAccessAllowed("setPerBlockCapUsd(uint256)");
    emit PerBlockCapUpdated(perBlockCapUsd, newCap);
    perBlockCapUsd = newCap;
}

/// @notice Updates the absolute USD slippage threshold above which
AbnormalSlippage is emitted
/// @param newThreshold New threshold value (1e18-scaled)
/// @custom:event SlippageEventUsdUpdated emits on success
/// @custom:access Restricted by ACM
function setSlippageEventUsd(uint256 newThreshold) external override
{
    _checkAccessAllowed("setSlippageEventUsd(uint256)");
    emit SlippageEventUsdUpdated(slippageEventUsd, newThreshold);
    slippageEventUsd = newThreshold;
}
```

Recommendation	Consider reordering the logic so that <code>_enforceCapAndDetectSlippage</code> , or at least an equivalent pre check on the projected <code>usdIn</code> , runs before the router call and the subsequent transfers.
Status	Acknowledged. The post-swap ordering is an intentional design trade-off. The <code>ResilientOracle</code> uses off-chain push feeds unaffected by on-chain swaps; using post-swap balance deltas as <code>actualAmountIn</code> is necessary for accurate measurement of fee-on-transfer tokens; <code>executeBuyback</code> is ACM-gated so gas cost on a reverting transaction falls on the operator only; and slippage detection is event-only with no on-chain consequence regardless of position.

[I03] Missing input validation checks between `perBlockCapUsd` and `dailyCapUsd`

Contract	<code>TokenBuyback.sol</code>
Severity Level	Informational
Description	Description The <code>TokenBuyback</code> contract does not enforce any relationship between <code>perBlockCapUsd</code> and <code>dailyCapUsd</code>. If <code>perBlockCapUsd</code> is configured to a value greater than <code>dailyCapUsd</code>, the per block cap becomes dead code because any swap that fits under the daily limit will automatically fit under the per block limit. This weakens the layered protection that the two caps are intended to provide and increases the risk of operator misconfiguration going undetected. <pre> JavaScript /// @notice Updates the daily USD cap on tokenIn consumption /// @param newCap New cap value (1e18-scaled) /// @custom:event DailyCapUpdated emits on success /// @custom:access Restricted by ACM function setDailyCapUsd(uint256 newCap) external override { _checkAccessAllowed("setDailyCapUsd(uint256)"); emit DailyCapUpdated(dailyCapUsd, newCap); dailyCapUsd = newCap; } /// @notice Updates the per-block USD cap on tokenIn consumption /// @param newCap New cap value (1e18-scaled) /// @custom:event PerBlockCapUpdated emits on success /// @custom:access Restricted by ACM function setPerBlockCapUsd(uint256 newCap) external override { _checkAccessAllowed("setPerBlockCapUsd(uint256)"); emit PerBlockCapUpdated(perBlockCapUsd, newCap); perBlockCapUsd = newCap; } </pre>

Recommendation	Consider including a check of <code>perBlockCapUsd <= dailyCapUsd</code> inside <code>setPerBlockCapUsd</code> and initialize, reverting on violations. Alternatively, if the relationship is intentionally left to operator discretion, document explicitly that maintaining <code>perBlockCapUsd <= dailyCapUsd</code> is the operator's responsibility so that the rationale is clear to reviewers and integrators.
Status	Fixed in 0c90ac7 . The entire per-block cap mechanism (<code>perBlockCapUsd</code> , <code>setPerBlockCapUsd</code> , <code>PerBlockCapExceeded</code> , <code>PerBlockCapUpdated</code>) was removed in commit <code>0c90ac7</code> , eliminating the misconfiguration risk entirely.

[I04] Faulty oracle disables cap and slippage detection in `_enforceCapAndDetectSlippage`

Contract	TokenBuyback.sol
Severity Level	Informational
Description	Description The <code>_enforceCapAndDetectSlippage</code> function in <code>TokenBuyBack</code> reads <code>priceIn</code> and <code>priceBase</code> from the oracle for <code>tokenIn</code> and <code>BASE_ASSET</code> and then computes <code>usdIn</code> and <code>usdOut</code> without verifying the oracle status. If <code>priceIn/priceOut</code> is ever 0, the <code>RESILIENT_ORACLE</code> will revert with an invalid price, preventing the buyback execution from occurring. <pre> JavaScript function _enforceCapAndDetectSlippage(address tokenIn, uint256 actualAmountIn, uint256 amountOut) internal { ResilientOracleInterface oracle = ResilientOracleInterface(RESILIENT_ORACLE); uint256 priceIn = oracle.getPrice(tokenIn); uint256 priceBase = oracle.getPrice(BASE_ASSET); uint256 usdIn = (actualAmountIn * priceIn) / 1e18; uint256 usdOut = (amountOut * priceBase) / 1e18; </pre>
Recommendation	Likely no fix required, consider documenting this scenario during an emergency faulty oracle scenario.
Status	Acknowledged. Under the Venus mainnet <code>ResilientOracle</code> configuration, a <code>BoundValidator</code> and pivot/fallback chain reject zero or out-of-band prices by reverting rather than propagating them to <code>TokenBuyback</code> . A faulty oracle therefore results in a liveness halt on <code>executeBuyback</code> — not a cap or slippage bypass. <code>forwardBaseAsset</code>

is unaffected as it does not invoke the oracle. No code change is required.

[I05] swap tokenIn donations via possible router callback bypasses the daily and per block caps in executeBuyback

Contract TokenBuyback.sol

Severity Level **Informational**

Description

Description

As a fix towards [I04], the TokenBuyback.executeBuyback function derives actualAmountIn from the difference between the tokenIn balance before and after the router call. If any code path executed during the router (e.g. allowing a callback or multicall) call deposits an additional non-zero amount of tokenIn into the contract, the post call balance grows by these amount relative to the genuine swap consumption.

JavaScript

```
function executeBuyback(
    address tokenIn,
    uint256 amountIn,
    uint256 minAmountOut,
    uint256 deadline,
    address router,
    bytes calldata routerCalldata,
    address comptroller
) external override nonReentrant {

    _checkAccessAllowed("executeBuyback(address,uint256,uint256,uint256,address,bytes,address)");
    _validateBuyback(tokenIn, amountIn, deadline, router);

    // Measure BASE_ASSET and tokenIn balances directly on this
    contract so the
    // delta reflects actual swap consumption regardless of any
    donation surplus
    // sitting outside the assetsReserves watermark. Router calldata
    MUST send
    // BASE_ASSET output here.
    uint256 baseAssetBalanceBefore =
    IERC20Upgradeable(BASE_ASSET).balanceOf(address(this));
    uint256 tokenInBalanceBefore =
    IERC20Upgradeable(tokenIn).balanceOf(address(this));

    IERC20Upgradeable(tokenIn).forceApprove(router, amountIn);
    router.functionCall(routerCalldata);
    IERC20Upgradeable(tokenIn).forceApprove(router, 0);
```

```

        uint256 tokenInBalanceAfter =
IERC20Upgradeable(tokenIn).balanceOf(address(this));
        uint256 amountOut =
IERC20Upgradeable(BASE_ASSET).balanceOf(address(this)) -
baseAssetBalanceBefore;
        uint256 actualAmountIn = tokenInBalanceBefore -
tokenInBalanceAfter;

        if (amountOut < minAmountOut) {
            revert SlippageExceeded(minAmountOut, amountOut);
        }

        if (amountOut != 0) {
            IERC20Upgradeable(BASE_ASSET).safeTransfer(DESTINATION,
amountOut);
        }

        assetsReserves[tokenIn] = tokenInBalanceAfter;
        assetsReserves[BASE_ASSET] =
IERC20Upgradeable(BASE_ASSET).balanceOf(address(this));

        _enforceCapAndDetectSlippage(tokenIn, actualAmountIn, amountOut);

        emit BuybackExecuted(tokenIn, actualAmountIn, amountOut, router,
comptroller);
    }

```

As a result, actualAmountIn will be overestimated and usdIn will be underestimated as the deposit would be considered as being consumed for the swap. The cap accounting in _enforceCapAndDetectSlippage is therefore undercharged by the token deposit worth of USD on every swap, possibly bypassing the window and block usd capping checks.

The donation can be triggered by mechanisms that execute during the router call, such as a DEX swap callback, a hooked token's tokensReceived handler, or a multi hop route that passes through a contract the attacker controls. However, this would require compromised admin permissions for Venu's own cron job which is the only address allowed to perform buyback executions

Recommendation	Likely no fix required, consider ensuring safety measures in place to prevent Venus cron job permissions from being compromised and ensure only trusted routers are whitelisted to prevent such callback attacks from occurring.
-----------------------	--

Status	Acknowledged. Exploiting this requires a compromised ACM-permitted operator, an allowlisted router with a mid-swap callback path, and a route through an attacker-controlled contract — all simultaneously. The current allowlisted router (PancakeSwap V2) has no callback path on swapExactTokensForTokens, and the
---------------	---

attacker must donate tokenIn out-of-pocket as the bypass cost. There is no viable runtime check to enforce "no-callback" as a router property on-chain. Existing mitigations — ACM-gated executeBuyback and governance-only setAllowedRouter — are sufficient given the attack prerequisites.

[I06] Reducing dailyCapUsd Below the Current Accumulator Value Temporarily Bricks executeBuyback

Contract TokenBuyback.sol

Severity Level **Informational**

Description

Description

setDailyCapUsd updates dailyCapUsd without resetting or clamping `usedConsumedInWindow`. When governance lowers `dailyCapUsd` below the current `usedConsumedInWindow`, the decayed accumulator value alone already exceeds the new cap, causing every subsequent `executeBuyback` to revert with `DailyCapExceeded` until natural leaky-bucket decay brings the level below the new cap. There is no external setter for `usedConsumedInWindow`, so governance has no direct means to unblock the function.

JavaScript

```
// TokenBuyback - setDailyCapUsd
function setDailyCapUsd(uint256 newCap) external override {
    _checkAccessAllowed("setDailyCapUsd(uint256)");
    ensureNonzeroValue(newCap);
    emit DailyCapUpdated(dailyCapUsd, newCap);
    dailyCapUsd = newCap;
    // usedConsumedInWindow is neither reset nor clamped
}

// TokenBuyback - _enforceCapAndDetectSlippage
uint256 decayed = elapsed >= WINDOW ? 0 : (usedConsumedInWindow *
(WINDOW - elapsed)) / WINDOW;
uint256 newConsumed = decayed + usedIn;
if (newConsumed > dailyCapUsd) {
    revert DailyCapExceeded(newConsumed, dailyCapUsd);
}
```

Lock duration scales with the reduction magnitude (assuming `usedConsumedInWindow` = `dailyCapUsd` = 30,000): reducing to 20,000 locks for ~8 hours; reducing to 5,000 locks for ~20 hours; reducing to near zero locks for up to ~24 hours.

Impact:

In scenarios requiring an urgent cap reduction — security incident response or risk tightening — a governance action can unintentionally render `executeBuyback` completely unavailable for up to 24 hours with no direct recovery path. User funds are not at risk, but normal protocol operations may be disrupted during time-sensitive situations.

Recommendation Clamp `usdConsumedInWindow` to the new cap inside `setDailyCapUsd` so the reduction takes effect immediately, or explicitly document this behaviour in the NatSpec and require governance to verify the current accumulator state before reducing the cap.

JavaScript

```
if (usdConsumedInWindow > newCap) {  
    usdConsumedInWindow = newCap;  
}
```

Status Fixed as recommended in commit [3e9caae](#).
